



UNIVERSIDAD DE GRANADA

TRABAJO FIN DE GRADO

Olivaris - Plataforma de gestión colaborativa para el olivar

Realizado por
Jorge Cano Melero



Para la obtención del título de
Grado en Ingeniería Informática

Dirigido por
Juan Luis Jiménez Laredo

En el departamento de
Dpto. de Ingeniería de Computadores, Automática y Robótica

Convocatoria de junio, curso 2025/26

Agradecimientos

Olivaris - Plataforma de gestión colaborativa para el olivar

Jorge Cano Melero

Palabras clave: Backend, API Rest

Resumen:

HACER RESUMEN

Olivaris - Plataforma de gestión colaborativa para el olivar

Jorge Cano Melero

Keywords: Backend, API Rest, **COMPLETAR**

Abstract:

SAME THAN SUMMARY BUT IN ENGLISH

Yo, **Jorge Cano Melero**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 77021264Z, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Jorge Cano Melero

Granada, 25 de febrero de 2026

D. **Juan Luis Jiménez Laredo**, Profesor del Área de Arquitectura y Tecnología de Computadores del Departamento de Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Olivaris - Plataforma de gestión colaborativa para el olivar*** , ha sido realizado bajo su supervisión por **Jorge Cano Melero**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada, 25 de febrero de 2026.

El director: Juan Luis Jiménez Laredo

Índice general

1	Introducción	1
1.1.	Estructura de la memoria	1
2	Estado del arte	3
2.1.	Contexto	3
2.2.	Análisis de Softwares ERP para la producción del aceite de oliva	4
2.3.	Comparativa con el estado del arte	9
2.4.	Desarrollo de Software enfocado en Servicios Web	9
2.4.1.	Arquitecturas Software	10
2.4.2.	Comunicación y Servicios Web	16
2.4.3.	Ecosistema del Desarrollo Web	17
2.4.4.	Sistemas de almacenamiento	20
2.4.5.	Despliegue y estrategias	21
2.4.6.	Autenticación y autorización	23
2.4.7.	Elección de tecnologías	24
3	Especificación de requisitos	26
3.1.	Recopilación de información	26
	Bibliografía	29

Índice de figuras

Índice de tablas

1. Introducción

AÑADIR UNA INTRODUCCIÓN

1.1. Estructura de la memoria

La memoria va a estar dividida en los siguientes capítulos:

- **Capítulo 1: Introducción**

En este capítulo se va a dar una breve introducción del producto que se va a desarrollar en el presente Trabajo de Fin de Grado. Además, se van a describir los distintos capítulos en los que se dividirá la memoria.

- **Capítulo 2: Estado del arte**

En este capítulo se realizará un análisis del contexto técnico y social del problema, así como un análisis de productos software relacionados, herramientas existentes para su desarrollo e implementación así como librerías.

- **Capítulo 3: Objetivos y Alcance**

Aunque los objetivos hayan sido mencionados anteriormente, aquí se entrará en más detalle describiendo tanto el objetivo general como los objetivos específicos del trabajo.

- **Capítulo 4: Metodología**

En este capítulo se explica cómo se ha trabajado durante el desarrollo del proyecto, abarcando tanto las metodologías de desarrollo como posibles intervenciones con personas usuarias (entrevistas, tests de usabilidad, etc).

- **Capítulo 5: Análisis y requisitos**

En este capítulo se va a realizar un análisis del problema general y se van a plasmar los requisitos. Para su realización se realizarán perfiles de usuario, historias de usuario y se expondrán los requisitos funcionales y no funcionales del sistema.

- **Capítulo 6: Diseño del sistema y arquitectura**

El contenido de este capítulo contendrá una descripción de cómo ha sido diseñada la solución sin entrar en detalles del código. Para su realización, se va a dividir en distintas secciones: arquitectura de alto nivel, modelo de datos, diseño de la interacción e interfaz, diseño de API y diseño de servicios.

- **Capítulo 7: Implementación**

Este capítulo explicará cómo se ha materializado el diseño del sistema en código. Estará dividido en las siguientes secciones: descripción de los módulos, tecnologías utilizadas, decisiones técnicas y problemas encontrados.

- **Capítulo 8: Evaluación y experimentación**

En este capítulo se va a comprobar cómo de bien funciona la implementación del sistema. Para ello, se podrán realizar pruebas de usabilidad, de rendimiento, de carga o disponibilidad del sistema.

- **Capítulo 9: Conclusiones y trabajos futuros**

En este punto se procederá a comentar qué se ha conseguido finalmente, qué limitaciones se han encontrado durante el desarrollo del proyecto y qué caminos quedan abiertos para ampliar o mejorar lo que ya se ha conseguido de cara al futuro.

- **Capítulo 10: Referencias**

Referencias a los documentos de los que se ha obtenido información que ha sido utilizada en el presente documento.

- **Capítulo 11: Glosario**

El glosario va a recoger todos aquellos términos técnicos y de relevancia que se hayan usado en el presente documento, con la finalidad de aclarar aquellos conceptos que puedan ser confusos o no conocidos.

2. Estado del arte

En esta sección se va a proceder a realizar un estudio del estado del arte en el ámbito de la planificación de recursos empresariales (Enterprise Resource Planning o ERP) para el sector de la oliva. Se analizarán cuáles son sus funcionalidades y limitaciones, así como las tecnologías utilizadas para su desarrollo.

2.1. Contexto

El desarrollo de software ERP para el sector de la oliva surge de la necesidad de integrar y controlar procesos muy específicos propios de una cadena agroindustrial compleja, caracterizada por una elevada regulación legislativa y por unos costes de producción y transformación significativos. La correcta gestión de la información se convierte en un factor clave para garantizar la eficiencia operativa, el cumplimiento normativo y la rentabilidad económica de las explotaciones, cooperativas y empresas del sector.

En este contexto, España se posiciona como líder mundial en la producción de aceite de oliva, concentrando de media en los últimos años más del 40 % de la producción mundial. A gran distancia le siguen otros países productores como Italia, con aproximadamente un 9,7 %, Grecia con un 8,2 % y Túnez con un 6,1 %, lo que pone de manifiesto la relevancia estratégica del sector oleícola español tanto a nivel económico como tecnológico [1]. Esta posición de liderazgo implica, además, una mayor exigencia en términos de control, trazabilidad y calidad del producto final.

El proceso de producción de la oliva no puede considerarse lineal ni estándar, ya que intervienen múltiples fases interconectadas y con un alto grado de variabilidad. Desde la gestión de parcelas agrícolas y la recolección de la aceituna, pasando por la recepción, el almacenamiento en depósitos o bodegas, la molturación, el envasado y finalmente la comercialización, cada etapa genera información crítica que debe ser registrada y relacionada con el resto del proceso. Asimismo, en este sector conviven distintas entidades, como agricultores, cooperativas, almazaras, envasadoras y comercializadoras, cada una con funciones específicas pero dependientes entre sí, lo que incrementa la complejidad de la gestión de los datos.

A esta complejidad productiva se suma una regulación especialmente estricta, orientada a garantizar la seguridad alimentaria, la trazabilidad y la calidad del aceite de oliva. Normativas como el Reglamento CE 178/2002 obligan a disponer de sistemas que permitan identificar el origen del producto y reconstruir su recorrido en cualquier punto de la cadena. Además, los consejos reguladores, las denominaciones de origen, los criterios de clasificación del aceite según su categoría y los análisis físico-químicos y organolépticos, así como las certificaciones

de calidad y producción ecológica, imponen requisitos adicionales que deben ser gestionados de forma precisa y documentada.

Las soluciones ERP generalistas presentan importantes limitaciones a la hora de adaptarse a este contexto, ya que no contemplan de forma nativa conceptos fundamentales del sector oleícola como la gestión por campañas agrícolas, los rendimientos grasos, los lotes dinámicos, las mezclas de calidades, los trasiegos entre depósitos o el control de mermas. Esto provoca que su implantación requiera un elevado nivel de personalización, incrementando los costes, los plazos y el riesgo de fracaso del proyecto.

Como consecuencia de estas necesidades específicas, se hace imprescindible el desarrollo de software ERP diseñado específicamente para el sector de la producción de oliva. (Estos sistemas se caracterizan por estar orientados a modelos de gestión por campañas, permitir un control exhaustivo de la trazabilidad desde la parcela hasta el producto final, gestionar lotes de forma dinámica y automatizar la generación de informes oficiales exigidos por las administraciones y organismos reguladores. Además, facilitan el control de costes, la gestión de la calidad y la integración de todos los agentes implicados en la cadena productiva, permitiendo a cooperativas, agricultores y empresas maximizar el valor de su producción y mejorar su competitividad en el mercado. QUITAR ??) A continuación, se va a proceder a analizar los softwares que realizan esta labor para ver cómo resuelven la problemática mencionada anteriormente.

2.2. Análisis de Softwares ERP para la producción del aceite de oliva

Actualmente, podemos encontrar diversos softwares ERP que están directamente relacionados con la producción de oliva, otros que no se relacionan directamente pero se ubican dentro del sector agrícola y otros que son genéricos pero se pueden adaptar al sector. Vamos a proceder a comentar algunos de ellos:

- **Agroptima - Relación directa (<https://www.agroptima.com>)**

Agroptima es un Software de gestión agrícola cuyo objetivo es conseguir la simpleza a la hora de gestionar los cultivos y explotaciones por parte de agricultores y empresas agrícolas (según declara la propia empresa).

En términos generales destaca por proporcionar un cuaderno de campo al agricultor, permitirle llevar un control de los gastos y costes agrícolas (como los trabajadores) y saber en todo momento en qué estado se encuentra su finca.

Entrando en más detalle, entre sus funcionalidades se encuentran:

- Localización de campos y cultivos, consulta de superficies y códigos SIGPAC.
- Anotación de toda la información que el agricultor considere relevante

desde el móvil y sin cobertura. Entre esta información se destaca: trabajadores, maquinaria, horas trabajadas y productos.

- Toda actividad realizada y anotada quedará en el sistema y servirá para mejorar la toma de decisiones y aumentar la rentabilidad de la gestión de la finca.
- Las fincas o cultivos se podrán importar en la aplicación con un archivo Excel o Shapes, permitiendo visualizar las parcelas coloreadas según el tipo de cultivo y su superficie total.
- Gestión de roles, permitiendo que el usuario pueda gestionar aquellas actividades relacionadas con el.

Agroptima también destaca por ofrecer a sus clientes la posibilidad de generar un Cuaderno de Campo.

■ **AgriCloud - Relación directa (<https://www.agricloud.es>)**

AgriCloud es un Software de gestión agrícola diseñado para llevar un control de las fincas, trabajos, personal y facturación, pensada para agricultores, cooperativas, empresas de servicios agrícolas y trabajadores por campaña. AgriCloud destaca por ofrecer funcionalidades como las siguientes:

- Un cuaderno de campo digital según la normativa (muy importante como hemos mencionado anteriormente).
- Control horario y partes de trabajo desde el móvil.
- Facturación electrónica conectada con Hacienda, permitiendo llevar un control del personal y ver la rentabilidad real por finca o cultivo.
- Costes ingresos e informes automáticos en un solo lugar.
- Acceso desde cualquier dispositivo, tanto para móvil como tablet u ordenador.
- Generación de un cuaderno de campo digital, facilitando la gestión y el seguimiento de las actividades agrícolas de forma eficiente y precisa.

Como hemos comentado, AgriCloud está pensada para poder satisfacer las necesidades de varios sectores:

- Agricultores individuales: podrán registrar en cuestión de segundos sus tareas pendientes o realizadas, llevarán un control de costes e ingresos por cultivo (sabiendo cuánto cuesta y cuánto produce por cada parcela) y podrán generar informes de forma automática.
- Cooperativas y asociaciones: destaca por aportar funcionales como el registro y consulta de la información de cada miembro y su producción, supervisión en tiempo real de cada tarea y de los resultados de la campaña, generación de informes globales o por socio en PDF o Excel, permite la comunicación y coordinación entre el personal a través de un entorno digital único.

- Empresas de servicios agrícolas: las empresas podrán gestionar partes digitales por cliente o finca, convertir dichos partes en facturas electrónicas al instante, control de maquinaria y materiales (horas y costes asociados a cada servicio o cliente), ayuda en la toma de decisión gracias a la consulta de resultados por tipo de trabajo y campañas.
- Trabajadores por campaña: AgriCloud ofrece una gestión de las “cuadrillas” de trabajadores a través de un fichaje y control del horario digital, asignación por tareas por finca o cultivo, accesos personalizados (cada perfil está adaptado al rol del empleado) y ofrece un seguimiento en la nube, consiguiendo que toda la información se sincronice automáticamente.

■ **TuOlivar - Relación directa (<https://www.tuolivar.com>)**

TuOlivar es un Software de gestión agrícola para la producción del olivar. Se destacan principalmente por ofrecer las siguientes funcionalidades:

- Gestión de la explotación: permiten el control de fincas, parcelas, maquinaria, herramientas, mantenimiento de la maquinaria, trabajadores y campañas.
- Alta organización y uso desde cualquier ubicación: permite llevar un control de los trabajos realizados, control de gastos e ingresos y de la cosecha realizada, pudiendo agrupar la información por campañas, fincas y categorías.
- Gestión total: permite dar de alta a los clientes, almazaras, proveedores, notas, documentos o avisos, así como generar distintos informes para su control de costes y cosechas.

■ **Agroex (<https://agroex.es>)**

Agroex es un Software de gestión agrícola industrial y ganadería, enfocado en empresas grandes. Cuentan con un software adaptado a las necesidades específicas del sector, permitiendo optimizar recursos y ahorrar costes (tanto en campaña agrícola específica como a lo largo del año), gracias a la administración y gestión tanto de los cultivos como del personal. Además, cuenta con integraciones como AEMET o MAGRAMA, permitiendo el acceso a datos en tiempo real que permite ayudar en la toma de decisiones.

Agroex ofrece soluciones para distintos sectores, desde un enfoque en la explotación agrícola y consultoría agraria a otros como bodegas, viñedos y viveros.

Si analizamos las funcionalidades que ofrece este Software, Agroex destaca por:

- Administración y gestión de explotaciones agrícolas y agroindustriales: permite la gestión de cultivos y administración de explotaciones a través de cuadernos de campo, gestión de plazos, riegos y recursos hídricos.
- Conectividad con entidades como MAGRAMA o AEMET para obtener información meteorológica precisa y en tiempo real.
- Gestión de costes.

- Gestión contable, generando de manera automática los apuntes contables para cada movimiento realizado.
- Gestión de almacén, controlando el stock y gestión de mercancías.
- Gestión documental, incluyendo la gestión de maquinaria, productos fitosanitarios y gestión de personal.
- Gestión económica y tesorería, generando informes completos y exactos de la liquidez y resultados económicos.
- Gestión de producción, a través del control de las actividades de fabricación, producción y manufactura.
- Gestión eficiente de los recursos hídricos.
- Recursos humanos, gracias a una conexión con la Seguridad Social.
- Facturación, ofreciendo herramientas para gestionar presupuestos, seguimiento comercial y factura electrónica.

■ **Galdón Software (Enfoque en Cooperativas)**
(<https://www.galdon.com/erp-almazaras-enzasadoras-aceite>)

Galdón Software es una empresa de ingeniería que ofrece soluciones software de gestión integrales para todo tipo de empresas e instituciones. Ofrecen servicios de consultoría y soluciones centralizadas para áreas de distintos negocios: ERP y CRM, Contabilidad y Finanzas, SGA, Apps, E-commerce y Recursos Humanos (RRHH).

Dentro de los sectores en los que operan, desarrollan Software ERP para almazaras y envasadoras de aceite. Con este ERP permiten llevar un control de la trazabilidad, producción y distribución del aceite y está pensado para cubrir las necesidades de cualquier empresa de aceite (oliva, girasol, soja, etc), almazara y envasadora.

Entre sus características destacan:

- Planning de entrada y salida de mercancías.
- Control de necesidades de materias primas.
- Generación automática de partes de envasado y planificación asistida.
- Business Intelligence: cuentan con herramientas que permiten mostrar comparativas gráficas de ventas y beneficios y análisis de ventas.
- Gestión de cosecheros y puestos de compra.

Empresas reconocidas en el sector como Maeva (Granada) y Aceites Vallejo trabajan con este Software.

■ **Campogest (<https://www.campogest.com>)**

Campogest es un Software diseñado por la empresa Hispatec y destinado para los Técnicos Agrícolas, permitiéndoles realizar una gestión de sus actividades a través del móvil o tablet.

Esta aplicación destaca por ofrecer las siguientes funcionalidades:

- Gestión de fincas, cultivos, previsión meteorológica, tratamientos fitosanitarios y recomendaciones personalizadas a través del móvil.
- Generación de un cuaderno de campo teniendo en cuenta las recomendaciones en materia de Normativa Legal y Protocolos de Calidad.
- Generación de planes de abonado y notificaciones a través del correo electrónico al agricultor.
- Posibilidad de generar la finca o parcela a través de la cartografía SIGPAC, pudiendo asignar un propietario. Gracias a esta funcionalidad los agricultores pueden dar de alta sus fincas y cultivos desde el mapa.

■ **Visual NACERT (<https://visualnacert.com>)**

Visual Nacert es una empresa que diseña soluciones digitales de gestión para empresas del sector agroalimentario, agricultores e industria. Cuentan con una aplicación agrícola, a la que se puede acceder a través de móvil o tablet, con o sin cobertura.

Desde esta aplicación, permiten al agricultor/cliente realizar las funciones:

- Gestión de costes insumos, mano de obra y maquinaria, generando gráficos a partir de estos datos e informes por parcela.
- Gestión del equipo de trabajo y control de stock.
- Gestión del Cuaderno de Campo Digital SIEX (de acuerdo a la normativa).
- Generación de informes de sostenibilidad.

Además de las funcionalidades anteriores, también ofrecen otras más centradas en la ayuda de toma de decisión al agricultor. Entre estas funcionalidades destacan:

- Informes de ayuda a la decisión basados en el análisis de datos del negocio y actividades realizadas en campo.
- Análisis de la finca / parcela para ayudar en la toma de decisiones estratégicas sobre qué cultivar.
- Herramienta de IA que predice las necesidades de riego del cultivo y ofrece recomendaciones para una gestión del agua eficiente.

■ **AM System (<https://amsystem.es>)**

AM System es una empresa que desarrolla software de gestión empresarial que ayuda a las empresas tanto en la gestión como en la toma de decisiones. Dentro de las soluciones que ofrecen, cuentan con un software de gestión para las almazaras.

Este software se basa en facilitar el trabajo diario en una almazara a través de la automatización de las tareas administrativas y de gestión. Entre sus

características destacan: seguimiento del producto (entrada aceituna hasta su comercialización), gestión de fincas, control de depósitos y almacenes, liquidaciones y generación automática de cargos (entre otras).

- **BlueAiOlive**

“BlueAiOlive es un software de gestión para almazaras y envasadoras de aceite, diseñado para cubrir todas las necesidades de gestión del sector agrícola”. Así es como la empresa BlueAiCor define su ERP, destacando que a parte de cubrir las funcionalidades básicas de un ERP, tales como control y administración empresarial, también cuentan con módulos avanzados para la producción, envasado y comercialización del aceite de oliva.

Para poder satisfacer dichas funcionalidades, cuentan con una herramienta integral que centraliza procesos, optimiza recursos y ofrece una visión global del negocio. De esta forma las empresas pueden tener un control total de cada etapa del proceso productivo.

Entrando en el apartado del propio software, BlueAiOlive ofrece una app para oleicultores, donde se encontrará toda la información centralizada y disponible para el cliente a través de una interfaz sencilla e intuitiva. A través de ella, la empresa podrá llevar un control de la contabilidad, control del stock del almacén, facturación y liquidaciones, gestión de salidas de aceite, gestión de rendimiento de laboratorio (entre otras más).

2.3. Comparativa con el estado del arte

Rellenar con la información de qué ofrece mi producto en comparación con lo que se ha analizado que hay en el mercado

2.4. Desarrollo de Software enfocado en Servicios Web

Históricamente, el desarrollo de servicios web se fundamentó en la Arquitectura Orientada a Servicios (SOA). Como indicaban Ortiz y Riestra (2009) [2], el reto principal era lograr la interoperabilidad entre sistemas heterogéneos mediante protocolos robustos como SOAP y el uso de XML para el intercambio de datos. En aquel momento, el foco estaba en permitir que terminales con capacidades limitadas pudieran acceder a lógica de negocio compleja residente en servidores.

No obstante, en la última década este paradigma ha evolucionado drásticamente. La rigidez de los protocolos basados en XML ha dado paso a la arquitectura REST (Representational State Transfer) y al formato JSON, que se han consolidado como el estándar de facto por su ligereza y facilidad de consumo, especialmente en dispositivos móviles.

En la actualidad, han emergido nuevos paradigmas que complementan y, en muchos casos, sustituyen a las arquitecturas monolíticas tradicionales. Destacan especialmente los microservicios y las arquitecturas orientadas a eventos (explicados a continuación), soluciones que han surgido como respuesta a la necesidad de construir sistemas con una mayor escalabilidad, flexibilidad y resiliencia ante grandes volúmenes de datos.

A continuación, se desglosarán los patrones de arquitectura software modernos, formatos de intercambio de datos y servicios web, ecosistemas de desarrollo web, desarrollo de aplicaciones móviles y por último, infraestructura y seguridad. Este análisis permitirá examinar de manera detallada cómo han evolucionado los servicios web hasta alcanzar los estándares de eficiencia y robustez contemporáneos.

2.4.1. Arquitecturas Software

La arquitectura de software ha evolucionado progresivamente hasta convertirse en un ecosistema adaptable. El crecimiento exponencial en el desarrollo de servicios web y la necesidad de ofrecer soporte a aplicaciones móviles con alta disponibilidad han impulsado la aparición de nuevos enfoques y modelos arquitectónicos. En este contexto, la transición de sistemas centralizados (arquitecturas monolíticas) hacia sistemas distribuidos ha surgido de manera natural, motivada por la necesidad de desplegar funcionalidades de forma independiente y escalable.

De acuerdo con Richards y Ford [3], las arquitecturas de software se pueden clasificar principalmente en dos categorías: monolíticas y distribuidas.

A continuación, se describen los estilos arquitectónicos más relevantes siguiendo la clasificación propuesta por los autores mencionados.

Arquitectura Monolítica

Es un tipo de arquitectura de software caracterizada principalmente por contener el código en una sola unidad de despliegue. Dentro de este tipo de arquitectura se pueden encontrar los siguientes tipos:

- **Arquitectura en capas**

Se utiliza como punto de partida ya que es el patrón más común y tradicional en el desarrollo de software. Este tipo de arquitectura se organiza en capas horizontales, donde cada una tiene un rol específico dentro de la aplicación (por ejemplo, presentación y lógica de negocio). Aunque no hay un número fijo de capas, el estándar suele ser de cuatro:

- **Capa de Presentación:** representa la interfaz de usuario con la que se interacciona y es la encargada de manejar las peticiones que son realizadas al servidor.
- **Capa de Negocio:** contiene las reglas y la lógica del dominio de la aplicación.

- Capa de Persistencia: es la encargada de manejar los datos del sistema (interacción con la base de datos).
- Capa de Base de Datos: corresponde al motor de almacenamiento de datos físico, es decir, la propia base de datos.

Una de las características más relevantes de este tipo de arquitectura es que las capas están aisladas. Esto significa que un cambio en una capa no debería afectar a las demás, siempre que la interfaz (el contrato) entre ellas se mantenga. En este punto, se puede diferenciar entre capas de tipo “cerrada”, que sería aquella que solo se puede comunicar con una inmediatamente inferior (tipo de capa por defecto), o de tipo “abierta”, la cual permite que capas superiores puedan saltársela para acceder directamente a capas inferiores.

Generalmente, este estilo se implementa como una única unidad de despliegue (monolito), lo que simplifica la gestión inicial pero dificulta la escalabilidad individual de componentes.

Valorando las características arquitectónicas se destaca:

- Costo y simplicidad: es una arquitectura muy fácil de entender y barata de implementar inicialmente.
- Facilidad de desarrollo: al ser un estándar tan conocido, los equipos pueden empezar a trabajar rápidamente.
- Escalabilidad y Elasticidad: esta característica es, quizás, su mayor debilidad y esto ocurre porque al ser monolítica, no va a permitir un escalado de las partes específicas del sistema de forma independiente.
- Tolerancia a fallos: esta es otra debilidad que presenta, ya que si se produce un fallo en una de las capas (por ejemplo, base de datos), el fallo va a afectar a toda la aplicación.

Como conclusión, se puede destacar que esta arquitectura es ideal para aplicaciones pequeñas y sencillas donde el presupuesto es limitado y la complejidad no justifica sistemas distribuidos más costosos.

■ **Arquitectura Pipeline**

Este estilo de arquitectura se basa en el principio de computación de una secuencia de transformaciones. Su estructura es lineal y se compone de dos elementos principales: *pipes* (tuberías) y *filters* (filtros). Por un lado, los *pipes* representan los canales de comunicación entre las etapas de procesamiento, suelen ser unidireccionales y su función es puramente el transporte de datos de un filtro a otro; mientras que por el otro lado, los *filters* son componentes independientes que realizan una tarea específica de procesamiento o transformación de los datos que reciben del *pipe*. Además, éstos son autónomos y no deben conocer la existencia de otros filtros en la cadena.

Continuando con los filtros, éstos se pueden clasificar en distintos tipos según su rol funcional dentro de la topología:

- *Producer (source)*: es el punto de inicio que genera los datos o los lee.
- *Transformer*: es el encargado de realizar transformaciones de datos.
- *Tester*: valida o filtra los datos basándose en criterios específicos.
- *Consumer (Sink)*: destino final donde terminan los datos.

En cuanto a sus características, esta arquitectura destaca por un alto desacoplamiento, ya que al ser los filtros independientes, es fácil intercambiar, añadir o modificar etapas en el pipeline sin afectar al resto del sistema. Además, este estilo es “*stateless*” entre filtros, es decir, cada filtro procesa lo que recibe y lo pasa al siguiente sin mantener un estado global complejo.

Valorando sus características arquitectónicas más relevantes, se clasifican en:

- Costo y simplicidad: es un estilo relativamente económico y fácil de implementar para los casos de uso adecuados.
- Facilidad de desarrollo: la separación clara de responsabilidades en filtros independientes facilita mucho el trabajo de codificación.
- Escalabilidad: al ser generalmente una arquitectura monolítica, es difícil de escalar de forma elástica.
- Tolerancia a fallos: si un filtro o una tubería falla, todo el proceso suele detenerse, lo que lo hace poco resiliente.

Como conclusión, esta arquitectura es ideal para aplicaciones que requieren un procesamiento de datos secuencial y discreto, como herramientas de procesamiento de texto, compiladores o sistemas de extracción, transformación y carga (ETL) sencillos.

■ **Arquitectura Microkernel**

También conocida como *plug-ins*, es un estilo de arquitectura diseñado para productos software que se pueden instalar (como navegadores o IDEs). Su estructura se divide en dos componentes principales: *Core System* (sistema central), el cual contiene toda la lógica mínima necesaria para que la aplicación funcione, encargándose de la ejecución básica y la gestión de los *plug-ins*; y *Plug-in Components*, que son módulos independientes entre sí con lógica adicional, características especiales o adaptadores personalizados para extender el sistema central.

En cuanto a su funcionamiento, el sistema central necesita saber qué *plug-ins* están disponibles. Para ello, utiliza un registro o catálogo que contiene información sobre el *plug-in*, cómo acceder a él y qué protocolos utiliza. Por parte del *plug-in*, para que este funcione debe cumplir con un contrato (interfaz) estándar definido por el sistema central.

Valorando sus características arquitectónicas más relevantes, se clasifican en:

- Costo y simplicidad: es relativamente sencillo de implementar y muy eficiente en términos de costo para productos que requieren

extensibilidad.

- Evolutividad: es su mayor fortaleza ya que permite añadir funcionalidades de forma aislada y rápida a través de plug-ins.
- Facilidad de prueba (testability): los plug-ins se pueden probar de forma aislada del sistema central.
- Escalabilidad y tolerancia a fallos: depende de la implementación, pero al ser frecuentemente monolítico, no escala tan bien como los sistemas microservicios.

Como conclusión, es una arquitectura ideal para aplicaciones que necesitan evolucionar constantemente añadiendo nuevas funciones o lógica según el cliente o el caso de uso, sin tener que reescribir el núcleo del sistema.

Arquitectura Distribuida

En esta arquitectura, a diferencia de la monolítica, los componentes van a estar separados y se conectarán a través de protocolos de acceso remoto. Entre sus tipos, se pueden destacar los siguientes:

■ Arquitectura basada en servicios

Este estilo se considera un híbrido que combina aspectos de la arquitectura monolítica y los microservicios, siendo una opción extremadamente popular para aplicaciones empresariales.

A diferencia de los microservicios, este estilo se basa en la creación de servicios de dominio de granularidad gruesa, refiriéndose a servicios más grandes que los microservicios y menos detallados, que encapsulan una funcionalidad de negocio más amplia y compleja. En una aplicación típica, suele haber entre 4 y 12 servicios, siendo cada servicio una unidad de despliegue independiente que contiene toda la lógica necesaria para un dominio de negocio específico. Sin embargo, la base de datos suele ser centralizada y compartida con todos (aunque si es necesario se puede “partir”), al igual que la interfaz de usuario, que hay una y es la que se comunica con los servicios.

Este estilo es calificado en el libro como uno de los más equilibrados, destacando:

- Agilidad y despleabilidad: al tener servicios independientes, es fácil realizar cambios y desplegarlos rápidamente.
- Facilidad de prueba: los servicios de dominio pueden probarse de forma aislada, lo que mejora la calidad del software.
- Costo y simplicidad: es mucho menos complejo y costoso de implementar que una arquitectura de microservicios pura.
- Escalabilidad y elasticidad: aunque es mejor que una arquitectura de capas, la base de datos compartida suele actuar como un cuello de botella que limita el escalado infinito.

- Tolerancia a fallos: el fallo de un servicio no detiene todo el sistema, pero un fallo en la base de datos compartida puede ser crítico.

Como conclusión, este estilo es usado para aplicaciones que necesitan un buen equilibrio entre agilidad, costo y escalabilidad, especialmente cuando el dominio del negocio tiene un acoplamiento semántico que dificultaría mucho el uso de microservicios.

■ **Arquitectura dirigida por eventos**

A diferencia del modelo tradicional basado en peticiones (*request-based*), donde un componente solicita datos de forma determinista, este modelo reacciona a situaciones que ocurren en el sistema (eventos). Se compone de componentes de procesamiento de eventos desacoplados que operan de manera asíncrona.

Se van a distinguir dos formas fundamentales de implementar este estilo:

- Topología de Broker (intermediario)
Esta topología está caracterizada por no tener un orquestador central, siendo el flujo de mensajes distribuido gracias a un *broker* ligero. Funciona mediante una cadena de eventos: un componente procesa un evento iniciador, publica lo que hizo como un ".evento de procesamiento", y otros componentes interesados reaccionan a él.
- Topología de Mediador
Esta topología se utiliza cuando se requiere un control sobre el flujo de trabajo. Para ello, se cuenta con un "Mediador de Eventos" que será el encargado, conociendo toda la lógica de negocio, de coordinar los pasos necesarios para procesar un evento inicial.

Este estilo está caracterizado por ser asíncrono, permitiendo de esta manera que el sistema responda al usuario casi instantáneamente mientras el procesamiento pesado ocurre en segundo plano. Esto evita bloqueos pero requiere del uso de técnicas (como colas de errores) que permitan gestionar fallos sin detener el sistema y que garanticen que los mensajes no se pierdan.

Según el libro, la calificación general que se da para este estilo es la siguiente:

- Escalabilidad y rendimiento: son sus mayores fortalezas, ya que al ser asíncrona y desacoplada permite procesar volúmenes masivos de datos.
- Agilidad y despleabilidad: al estar desacoplados, los componentes pueden cambiar y desplegarse con facilidad.
- Simplicidad y facilidad de prueba: es un estilo muy complejo de diseñar, implementar y, sobre todo, de probar debido a su naturaleza no determinista y distribuida.

Como conclusión, su uso es recomendado en aplicaciones que necesitan reaccionar en tiempo real a grandes flujos de información, como en sistemas de procesamiento de datos de telemetría.

■ **Arquitectura de microservicios**

Richards y Ford la definen como la .estrella brillante" del ecosistema actual, diseñada específicamente para abordar los problemas de escalabilidad y agilidad de los sistemas monolíticos.

Esta arquitectura se basa en el concepto de desacoplamiento extremo, tanto a nivel técnico como de dominio. A diferencia de otros estilos, su objetivo principal no es la reutilización (que a menudo se evita para no crear acoplamiento), sino la separación de intereses para permitir cambios rápidos. Aquí aparece una entidad clave que es el "microservicio", siendo una unidad física separada que se puede desplegar, escalar y actualizar sin afectar a los demás, y cuya funcionalidad será única.

Para garantizar el desacoplamiento de este estilo, cada microservicio posee sus propios datos. De esta manera, ningún otro servicio puede acceder directamente a la base de datos de otro; la comunicación debe ser a través de interfaces (APIs).

Entre sus componentes destacan:

- *API Gateway*: actúa como punto de entrada para los clientes, manejando tareas como métricas, seguridad y enrutamiento.
- *Sidecars*: componentes adjuntos a los servicios que manejan tareas transversales (comunicación, monitoreo) sin afectar a la lógica de negocio.
- *Service Mesh*: infraestructura para gestionar la comunicación entre todos los microservicios de forma consistente.

Aunque parezcan todo puntos a favor en esta arquitectura, es uno de los estilos más difíciles de implementar debido a la alta necesidad de gestión de los datos entre múltiples servicios y la complejidad de verificar que todo el ecosistema funcione correctamente. Además, requiere de conocimientos en áreas como Devops, automatización y monitoreo.

Según el libro, su calificación es la siguiente:

- Escalabilidad y elasticidad: es el mejor estilo para sistemas que necesitan crecer de forma masiva y dinámica.
- Agilidad y despleabilidad: Permite ciclos de entrega continuos.
- Tolerancia a fallos: el aislamiento asegura que el fallo de un servicio no derribe todo el sistema (siempre que se implementen patrones de resiliencia).
- Costo y simplicidad: es el estilo más caro y complejo de desarrollar, mantener y operar. No se recomienda para aplicaciones sencillas.

Como solución, la arquitectura de microservicios es muy útil para resolver problemas de gran escala, pero conlleva un impuesto de complejidad" que solo debe pagarse si las necesidades de negocio lo justifican.

2.4.2. Comunicación y Servicios Web

La comunicación entre componentes es el elemento que define la eficiencia de una arquitectura distribuida. Mientras que en el pasado la interoperabilidad se buscaba mediante protocolos estrictos basados en estándares corporativos, el auge de la web móvil ha impuesto la necesidad de protocolos de baja latencia y formatos de datos ligeros. Según Richards y Ford [3], el éxito de una arquitectura distribuida depende de la gestión de los contratos de comunicación y el ancho de banda disponible.

Esta interacción es posible gracias a las Interfaces de Programación de Aplicaciones (API). Entre sus tipos principales destacan:

- **REST (*Representational State Transfer*)**

Consolidado como el estándar de facto para servicios web. REST utiliza los métodos del protocolo HTTP para definir acciones sobre los recursos del sistema a través de *endpoints* (URLs).

Los principales verbos HTTP y su propósito son: [4]

- *GET*: solicita una representación de un recurso específico. Las peticiones que usan el método GET sólo deben recuperar datos.
- *PUT*: reemplaza todas las representaciones actuales del recurso de destino con la carga útil de la petición.
- *POST*: se utiliza para enviar una entidad a un recurso en específico, causando a menudo un cambio en el estado o efectos secundarios en el servidor.
- *DELETE*: borra un recurso en específico.
- *HEAD*: pide una respuesta idéntica a la de una petición GET, pero sin el cuerpo de la respuesta.
- *PATCH*: aplica modificaciones parciales a un recurso.
- *OPTIONS*: utilizado para describir las opciones de comunicación para el recurso de destino.
- *TRACE*: realiza una prueba de bucle de retorno de mensaje a lo largo de la ruta al recurso de destino.
- *CONNECT*: establece un túnel hacia el servidor identificado por el recurso.

Para confirmar el resultado de estas acciones, el servidor devuelve códigos de estado HTTP, tales como: 200 OK (éxito), 400 Bad Request (error del cliente) o 500 Internal Server Error (error del servidor).

- **GraphQL [5]**

GraphQL utiliza una sintaxis intuitiva que permite a los clientes enviar una única consulta GraphQL a una API y recibir exactamente los datos necesarios (en lugar de acceder a endpoints complejos con muchos parámetros). Esta

obtención de datos más eficiente puede mejorar el rendimiento del sistema y la facilidad de uso para los desarrolladores.

Es especialmente útil para crear APIs en entornos complejos con requisitos de interfaz que cambian rápidamente. Ni las API de REST ni las de GraphQL son inherentemente superiores; son herramientas diferentes que se adaptan a diferentes tareas.

- **gRPC** [6]

Es un marco de comunicación de alto rendimiento desarrollado por Google. Utiliza como sistema de comunicación el protocolo RPC (*Remote Procedure Call*) en lugar de texto, lo que lo hace significativamente más rápido que REST para la comunicación interna entre microservicios.

- **SOAP** [7]

SOAP (anteriormente conocido como Simple Object Access Protocol) es un protocolo ligero para el intercambio de información en entornos descentralizados y distribuidos. Es un protocolo basado en XML que define tres partes en todos los mensajes: *sobre*, que define una infraestructura para describir qué hay en un mensaje y cómo procesarlo; *reglas de codificación*, que expresa instancias de tipos de datos definidos por la aplicación, y *estilos de comunicación*, que pueden seguir el formato de llamada a procedimiento remoto (RPC) o el formato orientado a mensajes (Documento).

En cuanto al formato de intercambio de datos, Se ha producido una transición del uso de XML (extenso y costoso de procesar) al dominio absoluto de JSON (JavaScript Object Notation). JSON es un formato de texto ligero, fácil de leer para humanos y sumamente rápido de procesar por los motores modernos.

Finalmente, la comunicación entre sistemas va más allá del modelo tradicional de petición-respuesta de HTTP, existiendo otros protocolos para servicios que requieren inmediatez como **WebSockets**, proporcionando un canal de comunicación bidireccional constante entre el cliente y el servidor, y **Server-Sent Events**, permitiendo que el servidor envíe actualizaciones de forma automática al cliente sin que este tenga que solicitarlas repetidamente.

2.4.3. Ecosistema del Desarrollo Web

En este apartado se describen las principales tecnologías consideradas para el desarrollo del proyecto Olivaris. Estas se clasifican en dos grandes áreas: el Frontend (lado del cliente) y el Backend (lado del servidor).

Tecnologías para el frontend

El frontend comprende todo aquello con lo que el usuario interactúa directamente. En la última década, este campo ha evolucionado desde documentos estáticos hacia [Single Page Application](#) y [Progressive Web Apps](#) de gran complejidad. El objetivo actual es gestionar estados complejos, asegurar la accesibilidad y

optimizar el rendimiento bajo el paradigma de *Client-Side Rendering* (CSR) o enfoques híbridos.

Aunque hoy en día se utilizan muchas herramientas, la base de cualquier interfaz web descansa sobre tres pilares:

- **HTML5 (*HyperText Markup Language*)** [8]
Es el componente más básico de la Web. Define la estructura y el significado del contenido mediante etiquetas (marcas), permitiendo que el navegador interprete textos, imágenes y multimedia.
- **CSS3 (*Cascading Style Sheets*)** [9]
Lenguaje de estilos encargado de la presentación visual. Define cómo se renderizan los elementos en pantalla (diseño, colores, tipografías y adaptabilidad).
- **JavaScript** [10]
Si bien es más conocido como un lenguaje de *scripting* para páginas web, y es usado en muchos entornos fuera del navegador, JavaScript es un lenguaje de programación basada en prototipos, multiparadigma, de un solo hilo, dinámico, con soporte para programación orientada a objetos, imperativa y declarativa. Gracias a JavaScript, se ha conseguido que la web se convierta en una pieza interactiva y dinámica para el usuario.

Mención especial merece **TypeScript**, como un superset de JavaScript que añade tipado estático. Su uso mejora la detección de errores en tiempo de desarrollo y facilita el mantenimiento de bases de código extensas.

El desarrollo ha evolucionado del uso de "Vanilla JS" (JS puro) hacia marcos de trabajo (*frameworks*) que aumentan la eficiencia:

- **React** [11]
Librería basada en componentes reutilizables. Destaca por su flexibilidad y su capacidad para manejar interfaces de usuario altamente dinámicas mediante un DOM virtual.
- **Angular** [12]
Angular es un framework integral que utiliza TypeScript de forma nativa. Su arquitectura modular y su sistema de inyección de dependencias lo hacen ideal para aplicaciones empresariales escalables.

Para el ecosistema móvil, se han evaluado diversas estrategias que permiten alcanzar a usuarios de Android e iOS:

- **Flutter** [13]
Flutter es un framework de código abierto de Google para crear aplicaciones multiplataforma compiladas de forma nativa a partir de una única base de código, ofreciendo un alto rendimiento y una interfaz consistente.
- **React Native** [14]
React Native es una librería de JavaScript que permite construir aplicaciones nativas para Android e iOS. Proporciona componentes nativos que se asignan directamente a los bloques de construcción de la interfaz.

- **Kotlin** [15]

Kotlin es un lenguaje de programación de tipo estático utilizado para desarrollar aplicaciones móviles en Android, destacando por su seguridad y concisión.

- **Ionic** [16]

Ionic es un SDK (kit de desarrollo de software) de código abierto que permite crear aplicaciones multiplataforma utilizando una única base de código. La aplicación se construye una sola vez con tecnologías web (como HTML, CSS y JavaScript), y se despliega en cualquier dispositivo.

Tecnologías para el backend

El backend representa la capa de lógica de procesamiento y acceso a datos que reside en el servidor. El enfoque actual prioriza la escalabilidad, la capacidad de respuesta ante grandes volúmenes de datos y la exposición de servicios mediante APIs (*Application Programming Interfaces*) robustas, generalmente bajo el estándar REST o GraphQL.

Dentro de los lenguajes más utilizados para el desarrollo del backend, se encuentran:

- **Java**

Java se mantiene como un pilar en sistemas empresariales críticos debido a su robustez, tipado fuerte y excelente gestión de memoria a través de la JVM (*Java Virtual Machine*).

Uno de los frameworks de Java más utilizados es Spring Boot [17], siendo el framework de código abierto líder, que facilita el uso de marcos de trabajo basados en Java para crear microservicios y aplicaciones web autónomas.

En cuanto a sus ventajas, proporciona configuración automática de librerías, servidores embebidos (como Tomcat) y un ecosistema maduro para la creación de entornos de pruebas. Además, Aumenta drásticamente la productividad al eliminar la configuración manual repetitiva.

- **Python**

Es el lenguaje con mayor crecimiento actual debido a su sintaxis limpia y su dominio en Inteligencia Artificial y Ciencia de Datos, lo que facilita la integración de modelos de *Machine Learning* en el backend.

Python también puede ser útil para el desarrollo web, ofreciendo una amplia gama de frameworks como Django, Flask o FastApi.

En cuanto a Django [18], es un framework para desarrollo de aplicaciones con todo incluido "diseñado para el desarrollo rápido. Utiliza una arquitectura basada en módulos reutilizables, lo que reduce significativamente el tiempo de salida al mercado (*Time-to-Market*). Es la base de plataformas de gran escala como Instagram.

Por otro lado, FastApi [19], siendo una opción moderna y de alto rendimiento basada en el tipado de Python. Destaca por la generación automática de documentación técnica (OpenAPI/Swagger) y su soporte nativo para programación asíncrona, lo que lo hace ideal para APIs de baja latencia.

- **Node.js** [20]

Node.js no es un lenguaje en sí, sino un entorno de ejecución que permite ejecutar JavaScript en el servidor. Su arquitectura está basada en un modelo de E/S (entrada/salida) sin bloqueo y orientado a eventos. Es extremadamente eficiente para aplicaciones en tiempo real y servicios que manejan una gran cantidad de conexiones simultáneas, ofreciendo una alta escalabilidad horizontal.

Para su desarrollo web, cuenta con frameworks como **Express**, siendo un framework minimalista y flexible que ofrece libertad total al desarrollador, **NestJs**, que destaca por ser progresivo y modular (inspirado en Angular) que utiliza TypeScript por defecto, ideal para construir aplicaciones backend escalables y fáciles de mantener.

2.4.4. Sistemas de almacenamiento

El almacenamiento de datos ha evolucionado desde sistemas de archivos planos hacia Sistemas de Gestión de Bases de Datos (SGBD). En el desarrollo moderno, la elección de una base de datos no depende solo del volumen de información, sino de su estructura, la velocidad de consulta y el nivel de consistencia requerido. El estado del arte actual se divide principalmente en los modelos Relacionales (SQL) y No Relacionales (NoSQL).

Bases de Datos Relacionales (SQL) [21]

Este modelo organiza los datos en tablas con relaciones predefinidas. Cada tabla representa una entidad, estructurando la información en filas (registros) y columnas (atributos). Su principal fortaleza es la integridad referencial y el uso del lenguaje estandarizado SQL (*Structured Query Language*) para la manipulación de datos.

Estas bases de datos destacan por tener un esquema rígido (es necesario definir la estructura antes de insertar datos) y cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

Entre las más destacadas se encuentran: MySQL / MariaDB, PostgreSQL y Microsoft SQL Server.

Base de Datos NoSQL [22]

Diseñadas para modelos de datos específicos, las bases de datos NoSQL ofrecen esquemas flexibles que permiten una alta escalabilidad horizontal. Son ideales para aplicaciones modernas que requieren un desarrollo rápido e iterativo.

Se clasifican según su forma de almacenar la información:

- Orientadas a documentos: almacenan datos en formatos similares a JSON (como MongoDB).
- Clave - Valor: optimizadas para lecturas muy rápidas mediante una clave única (Redis).
- Orientadas a grafos: enfocadas en la relación entre nodos (como Neo4j).

Están caracterizadas por ofrecer escalabilidad horizontal mediante clústeres distribuidos y manejo eficiente de grandes volúmenes de datos no estructurados.

Bases de Datos en la Nube [23]

Representan la evolución del almacenamiento hacia el modelo *as-a-service*. Aunque pueden ser relacionales o NoSQL, se diferencian de las locales por aprovechar los beneficios de la computación en la nube:

- Escalabilidad y Agilidad: permiten aumentar los recursos de forma elástica según la demanda.
- Reducción de costes: eliminan la necesidad de gestionar hardware físico y mantenimiento de infraestructura.
- Alta disponibilidad: ofrecen réplicas automáticas y copias de seguridad gestionadas por el proveedor (como AWS RDS o Azure SQL).

2.4.5. Despliegue y estrategias

El despliegue de software (*deployment*) es el proceso crítico que transforma el código fuente en una aplicación funcional y accesible para los usuarios finales. En la actualidad, se concibe como un ciclo continuo y automatizado que garantiza la disponibilidad y estabilidad del sistema. Este proceso se compone de las siguientes etapas fundamentales:

- **Integración Continua (CI):** es la fase donde el código se compila, valida y somete a pruebas automáticas cada vez que un desarrollador realiza un cambio, asegurando la integridad de la base de código.
- **Entrega/Despliegue continuo (CD):** una vez validado el código, se realiza el envío automático a los entornos de prueba o producción.
- **Aprovisionamiento de infraestructura:** consiste en la creación y configuración de los recursos (servidores, bases de datos, redes) donde residirá la aplicación, idealmente mediante Infraestructura como Código (IaC).
- **Monitorización y *feedback*:** una vez en producción, la aplicación es vigilada para detectar errores en tiempo real, permitiendo un *rollback* (vuelta atrás) inmediato si se detectan anomalías.

Un aspecto fundamental es decidir que estrategia de despliegue se va a llevar a cabo cuando se realiza una aplicación y cómo se libera a los usuarios. Los tipos de estrategia pueden variar según las necesidades de la infraestructura y según la estrategia que se quiera seguir.

Según la infraestructura, el despliegue se puede realizar en servidores físicos propios (*on-premise*), a través de proveedores en la nube (como *AWS*), o ejecutándose a través de eventos, sin gestión directa de servidores (*serverless*). Por otro lado, según la estrategia de liberación que se siga, se encuentran los siguientes tipos: [24]

- *Rolling upgrade*
Se realiza una actualización secuencial del servicio, actualizando uno a uno los componentes. Este tipo de despliegue tiende a ser lento tanto en su ejecución como en su capacidad de volver atrás (*rollback*) y generalmente se emplea en combinación con otras estrategias.
- *Blue/Green*
Estrategia muy común que implica el uso de dos versiones del servicio: la versión nueva (*blue*) y la versión estable existente (*green*). Los usuarios continúan utilizando la versión verde mientras se prueba y evalúa la versión azul. Cuando se considera lista, los usuarios cambian a la versión azul. Si se detecta algún problema, es posible regresar a la versión verde.
- *Red/Black*
Similar a la estrategia blue/green, con la diferencia de que se desvía una pequeña porción del tráfico desde la versión negra (estable) hacia la nueva versión. Luego de evaluar su rendimiento, se realiza un desvío aún más grande del tráfico y, finalmente, se redirecciona todo el tráfico hacia la nueva versión. Se mantiene la versión negra en línea para permitir un *rollback* rápido si es necesario.
- *Dark Launcher*
Se implementa una nueva versión en paralelo a la versión estable existente. La particularidad aquí es que se duplica una parte del tráfico para observar cómo se comporta la nueva versión bajo una carga duplicada. Luego, se realiza un redireccionamiento parcial del tráfico hacia la nueva versión.
- *Canary Release*
Esta estrategia implica un despliegue parcial basado en el tiempo y el rendimiento de la nueva versión, consiguiendo de esta manera una implementación gradual y controlada.

En cuanto a las tecnologías y herramientas que se pueden utilizar para realizar el despliegue y gestionar una aplicación, se pueden clasificar en:

- Herramientas de contenerización, siendo las más utilizadas son Docker y Kubernetes. Por un lado, Docker [25] empaqueta software en unidades estandarizadas llamadas contenedores que incluyen todo lo necesario para que el software se ejecute, incluidas bibliotecas y herramientas de sistema, pudiendo ejecutar aplicaciones en cualquier entorno. Por otro lado, Kubernetes [26] facilita la automatización y la configuración declarativa,

además de administrar contenedores, orquestando la infraestructura de cómputo, redes y almacenamiento para que las cargas de trabajo de los usuarios no tengan que hacerlo.

- Herramientas de automatización, como GitHub Actions o Jenkins que ejecutan los flujos de trabajo de forma automática.
- Herramientas que permiten la definición de la infraestructura como código (IaC), como Terraform, que permiten definir y gestionar la infraestructura mediante archivos de configuración, facilitando la replicabilidad.

2.4.6. Autenticación y autorización

La seguridad es un apartado crítico en el desarrollo de cualquier sistema; es una responsabilidad transversal que se apoya en protocolos estandarizados para garantizar la interoperabilidad.

La autenticación (*AuthN*) se define como el proceso técnico de verificar que un usuario es quien dice ser, constituyendo la primera barrera de defensa. A continuación, se detallan los mecanismos más utilizados:

- **Basada en sesiones (formularios + cookies) [27]**

Es el método tradicional donde el usuario envía sus credenciales y, tras la validación, el servidor crea una sesión y devuelve una cookie al navegador. En cada petición posterior, el cliente envía dicha cookie para que el servidor valide el ID de sesión.

Es un método que destaca su alta seguridad en aplicaciones web simples y capacidad de revocación instantánea del acceso; sin embargo, presenta dificultades para escalar (debido al estado en el servidor).

- **Basada en tokens (JWT) [28]**

Es el estándar predominante en aplicaciones modernas y arquitecturas desacopladas. Tras la validación, el servidor genera un JSON Web Token (JWT) firmado digitalmente. El cliente lo almacena y lo incluye en la cabecera de cada petición para acceder a recursos protegidos.

Entre sus ventajas, destaca por una escalabilidad total (stateless) y compatibilidad nativa con aplicaciones móviles; sin embargo, presenta un riesgo de seguridad si el token es interceptado, ya que el acceso es válido hasta su expiración (difícil de revocar antes de tiempo).

- **OAuth 2.1 y OpenID Connect [29]**

OAuth 2.1 es un framework que permite el acceso delegado (una aplicación accede a datos de otra sin compartir contraseñas), mientras que OpenID Connect (OIDC) es una capa de identidad sobre OAuth que habilita el *Single Sign-on*.

Este sistema de autenticación destaca por su gran escalabilidad e integración con múltiples proveedores (Google, GitHub, etc.); sin embargo, presenta una mayor complejidad técnica en su implementación inicial.

- **Multifactor [30]**

Se trata de un mecanismo que combina diversos factores de verificación: conocimiento (PIN/contraseña), posesión (dispositivo móvil) e inherencia (biometría).

Como ventaja destaca por mitigar riesgos de robo de credenciales o tokens; aunque un atacante obtenga el JWT, carecería del segundo factor; sin embargo, Puede degradar la experiencia de usuario (UX) al añadir fricción y lentitud al proceso de entrada.

2.4.7. Elección de tecnologías

Tras el análisis de las opciones disponibles, se han seleccionado las siguientes tecnologías para el desarrollo del proyecto Olivaris, buscando un equilibrio entre rendimiento, seguridad y mantenibilidad.

- **Frontend: React Native**

El acceso a la plataforma será principalmente a través de dispositivos móviles, dada la naturaleza del trabajo agrícola. La movilidad y disponibilidad inmediata que ofrece un smartphone son claves para que los agricultores interactúen con el sistema a pie de campo.

Las razones de la elección han sido:

- Utiliza JavaScript/TypeScript, uno de los lenguajes con mayor comunidad y demanda laboral, superando actualmente el ecosistema de Flutter en oportunidades regionales.
- A diferencia de las soluciones híbridas, React Native utiliza componentes nativos reales de iOS y Android, respetando las guías de diseño y ofreciendo una experiencia de usuario fluida.
- El uso de Expo facilita la configuración del entorno (evitando dependencias complejas de Android Studio) y permite actualizaciones *Over-The-Air* (OTA), permitiendo corregir errores sin que el usuario tenga que descargar una nueva versión de las tiendas oficiales.
- El ecosistema de librerías es muy amplio. Si se necesita integrar un SDK de terceros (pagos, mapas, analíticas), es muy probable que ya exista un paquete oficial o muy probado para React Native.

- **Backend: Java con SpringBoot**

Para la lógica de servidor se ha optado por el ecosistema de Java, priorizando la robustez y la seguridad del sistema. Las razones por las que se ha elegido son las siguientes:

- El tipado estático de Java permite detectar la mayoría de los errores en tiempo de compilación. Esto ofrece una seguridad que lenguajes dinámicos como Python o JavaScript no pueden garantizar en la misma medida.

- Java maneja hilos de ejecución de forma eficiente, siendo superior en tareas que requieren un uso intensivo de CPU o procesamiento paralelo.
- Spring Boot ofrece una estructura de proyecto coherente y estandarizada. La gestión de acceso a datos y seguridad se realiza bajo patrones conocidos, facilitando que cualquier desarrollador se integre rápidamente en cualquier proyecto.
- Cuenta con *Spring Security*, uno de los marcos de seguridad más avanzados y granulares del mercado.
- El sistema está preparado para manejar transacciones ACID complejas. Además, la facilidad de Java para generar reportes en PDF y su compatibilidad con sensores IoT aseguran que el proyecto pueda crecer sin limitaciones técnicas.

Teniendo en cuenta lo anterior, aunque inicialmente la carga de procesamiento no sea muy alta, SpringBoot permite manejar las transacciones ACID de forma impecable y este proyecto, con una sola acción se van a desencadenar diferentes acciones “por debajo”. Además, Java cuenta con librerías muy potentes para generar PDFs y si en un futuro se deciden añadir funcionalidades extras, la escalabilidad es muy buena (integración con sensores IoT).

También lo he decidido así porque Java con SpringBoot es una tecnología muy demandada en el mercado laboral actualmente, por lo que considero que es un buen punto para empezar a aprenderlo.

■ **Base de datos: PostgreSQL**

Se ha optado por un modelo relacional (SQL) debido a la complejidad de las asociaciones y dependencias jerárquicas entre las entidades del dominio de Olivaris.

El uso de PostgreSQL en lugar de MySQL o MariaDB se debe a las siguientes razones:

- Existen relaciones jerárquicas, por lo que un modelo relacional es el más eficiente para garantizar la integridad de los datos cuando existen múltiples tablas interconectadas.
- PostgreSQL destaca sobre otros motores SQL por su soporte avanzado de tipos no estructurados ([JSONB](#)), permitiendo lo mejor de los dos mundos: la rigidez del SQL y la flexibilidad de NoSQL en campos específicos.
- Su estricto cumplimiento de los principios ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) garantiza que la información nunca se corrompa, incluso ante fallos del sistema.

3. Especificación de requisitos

Antes de empezar con el desarrollo e implementación del sistema es necesario realizar una fase previa correspondiente al análisis. En esta fase se van a realizar actividades que permitan extraer requisitos que va a tener el sistema con la finalidad de tener desde el inicio un planteamiento adecuado de las funcionalidades que se tendrán. A continuación se va a desarrollar como ha sido la fase de análisis y obtención de requisitos para el sistema

3.1. Recopilación de información

Para recopilar información se ha recurrido a la realización de entrevistas, análisis de documentos y consultas a través de correo electrónico con el personal adecuado para el fin a realizar.

- **Entrevistas**

Se han realizado varias entrevistas con la finalidad de ajustar el sistema lo máximo posible a lo que se necesita o sería interesante que hubiese en el sector laboral.

- **José (APELLIDOS) - Agricultor de (NOMBRE COOPERATIVA/EMPRESA)**

Con esta reunión se pudo ver cómo es el día a día de un agricultor que tiene varias fincas y cómo las gestiona. José mostró las distintas tablas que tiene creadas en Excel a través de las cuáles podía llevar un control exacto de qué actividad ha realizado en cada campaña y sobre qué finca en particular. Fueron varios los puntos planteados durante la reunión sobre qué podría tener la aplicación Olivaris. Entre ellos se destacaron:

- Funcionalidad de introducir un cuaderno de campo, siendo un punto muy interesante a tratar ya que el próximo enero de 2027 va a ser obligatorio. Sobre esta funcionalidad se ha realizado un análisis más profundo que será comentado a continuación.
- Funcionalidad sobre control de costes y gastos, además de los pagos realizados a los empleados contratados.
- Funcionalidad sobre gestionar parcelas y las actividades realizadas en cada una de ellas, de manera que se tenga una vista de todas las parcelas que son de su propiedad y que se puedan gestionar sus actividades.

Gracias a este reunión se pudo obtener una aproximación de qué tipo de tablas se tendrían en la base de datos y como se podrían relacionar entre ellas.

- **Juan Luis Jiménez Laredo - Profesor titular de la ETSIIT y Director del presente TFG**

Las reuniones realizadas con Juan Luis han servido para llevar una revisión continua del desarrollo del proyecto, con la finalidad de saber qué requisitos funcionales tendrá el sistema y qué tareas hay que priorizar. Además, contando también con experiencia en el sector de la oliva, ha podido dar una visión del sistema más precisa, siendo esto de gran ayuda para definir cuáles son las funcionalidades más relevantes de la aplicación.

Con la primera reunión realizada con Jose y con las realizadas con Juan Luis, se exploró la opción de introducir la funcionalidad del cuaderno de campo. El cuaderno de campo es hablar del cuaderno de campo, el servidor rea y cue, la api del gobierno y problemas que se han tenido para acceder

Anexo: Glosario

En esta sección se van a recoger las definiciones de términos que hayan sido utilizados en la memoria:

Application Programming Interface (API) : conjunto de reglas, protocolos y herramientas que permite que dos aplicaciones de software se comuniquen e intercambien datos entre sí de forma segura.

Single Page Application : es un tipo de aplicación web que ejecuta todo su contenido en una sola página, cargando el contenido HTML, CSS y JavaScript por completo al abrir la web. Al ir pasando de una sección a otra, solo necesita cargar el contenido nuevo de forma dinámica si este lo requiere, pero no hace falta cargar la página por completo.

Progressive Web Apps : es una aplicación web que, gracias a tecnologías modernas (HTML, JavaScript, CSS), ofrece una experiencia similar a una aplicación nativa, permitiendo su instalación en la pantalla de inicio, funcionar sin conexión y enviar notificaciones push, todo desde una única base de código y sin necesidad de tiendas de aplicaciones, combinando lo mejor de la web y las apps nativas.

Client-Side Rendering : técnica que permite al navegador del usuario descargar un HTML mínimo y luego usar JavaScript para construir y renderizar el contenido de la página web dinámicamente en el cliente.

Marcos de trabajo (frameworks) : conjunto estandarizado de herramientas, librerías, reglas y componentes reutilizables que actúa como esqueleto para desarrollar software, aplicaciones web o móviles de forma rápida y eficiente.

JSON Web Token : es un estándar abierto para la transmisión segura de la información entre dos partes (típicamente un cliente y un servidor). Cada JWT es firmado digitalmente para prevenir su manipulación y además, contiene *claims* (piezas de información) sobre el usuario o la sesión.

Single Sign-on : procedimiento de autenticación que habilita a un usuario determinado para acceder a varios sistemas con una sola instancia de identificación.

JSONB : formato binario para guardar JSON que permite indexar los datos.

Bibliografía

- [1] Aceite de las Valdesas. Principales países productores de aceite de oliva, 2026. URL https://www.aceitedelasvaldesas.com/faq/varios/produccion-aceite-de-oliva-por-paises/?srsltid=AfmB0orTk2Hc_z0941GBcuBYAxz1h0AVcWQ8fFockJPVzTscrF9gl97-. Accedido el 09 de enero de 2026.
- [2] Pablo Ortiz Sanchidrián y Lourdes Riestra Alba. Descubrimiento de servicios para la evolución dinámica de sistemas software mediante transformación de modelos, 2009. URL https://oa.upm.es/1387/1/PFC_PABLO_ORTIZ_LOURDES_RIESTRA.pdf. Accedido el 16 de enero de 2026.
- [3] Neal Ford Mark Richards. Fundamentals of software architectures, O'Reilly, 2020. ISBN 978-1-492-04345-4.
- [4] MDN Developer Resource. Métodos de petición http, 2026. URL <https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods>. Accedido el 19 de enero de 2026.
- [5] IBM. ¿qué es graphql?, 2026. URL <https://www.ibm.com/es-es/think/topics/graphql>. Accedido el 19 de enero de 2026.
- [6] Viewnext. ¿qué es el grpc? ventajas e inconvenientes, 2022. URL <https://www.viewnext.com/que-es-el-grpc-ventajas-e-inconvenientes/>. Accedido el 19 de enero de 2026.
- [7] IBM. Soap, 2021. URL <https://www.ibm.com/docs/es/radfw/9.7.0?topic=standards-soap>. Accedido el 19 de enero de 2026.
- [8] MDN Developer Resource. Html, 2026. URL <https://developer.mozilla.org/es/docs/Web/HTML>. Accedido el 20 de enero de 2026.
- [9] MDN Developer Resource. Css, 2026. URL <https://developer.mozilla.org/es/docs/Web/CSS>. Accedido el 20 de enero de 2026.
- [10] MDN Developer Resource. Javascript, 2026. URL <https://developer.mozilla.org/es/docs/Web/JavaScript>. Accedido el 20 de enero de 2026.
- [11] React dev. React, 2026. URL <https://es.react.dev/>. Accedido el 20 de enero de 2026.
- [12] Angular docs. Angular, 2026. URL <https://docs.angular.lat/>. Accedido el 20 de enero de 2026.
- [13] Flutter docs. Flutter, 2026. URL <https://esflutter.dev/>. Accedido el 20 de enero de 2026.

- [14] React Native docs. React native, 2026. URL <https://reactnative.dev/>. Accedido el 20 de enero de 2026.
- [15] Android developer. Kotlin, 2026. URL <https://developer.android.com/kotlin?hl=es-419>. Accedido el 20 de enero de 2026.
- [16] ESIC University. Ionic, 2025. URL <https://www.esic.edu/rethink/tecnologia/ionic-que-es-impacto-en-desarrollo-apps-c>. Accedido el 20 de enero de 2026.
- [17] Microsoft. Spring boot, 2026. URL <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-java-spring-boot#:~:text=Obt%C3%A9n%20una%20introducci%C3%B3n%20a%20Spring%20Boot%20en,que%20facilitan%20el%20desarrollo%20de%20aplicaciones%20Java>. Accedido el 20 de enero de 2026.
- [18] Amazon Web Services. Django, 2026. URL <https://aws.amazon.com/es/what-is/django/>. Accedido el 20 de enero de 2026.
- [19] FastApi docs. Fastapi, 2026. URL <https://fastapi.tiangolo.com/es/features/#fastapi-features>. Accedido el 20 de enero de 2026.
- [20] CTO en OpenWebinars Jesús Lucas Flores. Node.js, 2019. URL <https://openwebinars.net/blog/que-es-nodejs/>. Accedido el 26 de enero de 2026.
- [21] Microsoft. Bases de datos relacionales, 2026. URL <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-a-relational-database>. Accedido el 26 de enero de 2026.
- [22] Amazon Web Services. Bases de datos nosql, 2026. URL <https://aws.amazon.com/es/nosql/>. Accedido el 26 de enero de 2026.
- [23] Google Cloud. Bases de datos en la nube, 2026. URL <https://cloud.google.com/learn/what-is-a-cloud-database?hl=es-419>. Accedido el 26 de enero de 2026.
- [24] Sentries. Estrategias de despliegue: concepto y tipos, 2023. URL https://sentries.io/blog/estrategias-de-despliegue/#Tipos_de_despliegues. Accedido el 27 de enero de 2026.
- [25] Amazon Web Services. Docker, 2026. URL <https://aws.amazon.com/es/docker/>. Accedido el 27 de enero de 2026.
- [26] Documentación de Kubernetes. Kubernetes, 2026. URL <https://kubernetes.io/es/docs/concepts/overview/what-is-kubernetes/>. Accedido el 27 de enero de 2026.
- [27] MDN Developer Resource. Http cookies, 2026. URL <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Cookies>. Accedido el 29 de enero de 2026.

- [28] JWT documentation. Introduction to json web tokens, 2026. URL <https://www.jwt.io/introduction#what-is-json-web-token>. Accedido el 29 de enero de 2026.
- [29] auth0. ¿qué es oauth 2.0?, 2026. URL <https://auth0.com/es/intro-to-iam/what-is-oauth-2>. Accedido el 29 de enero de 2026.
- [30] Microsoft. ¿qué es la autenticación multifactor?, 2026. URL <https://support.microsoft.com/es-es/topic/-qu%C3%A9-es-la-autenticaci%C3%B3n-multifactor-e5e39437-121c-be60-d123-eda06bddf661>. Accedido el 29 de enero de 2026.